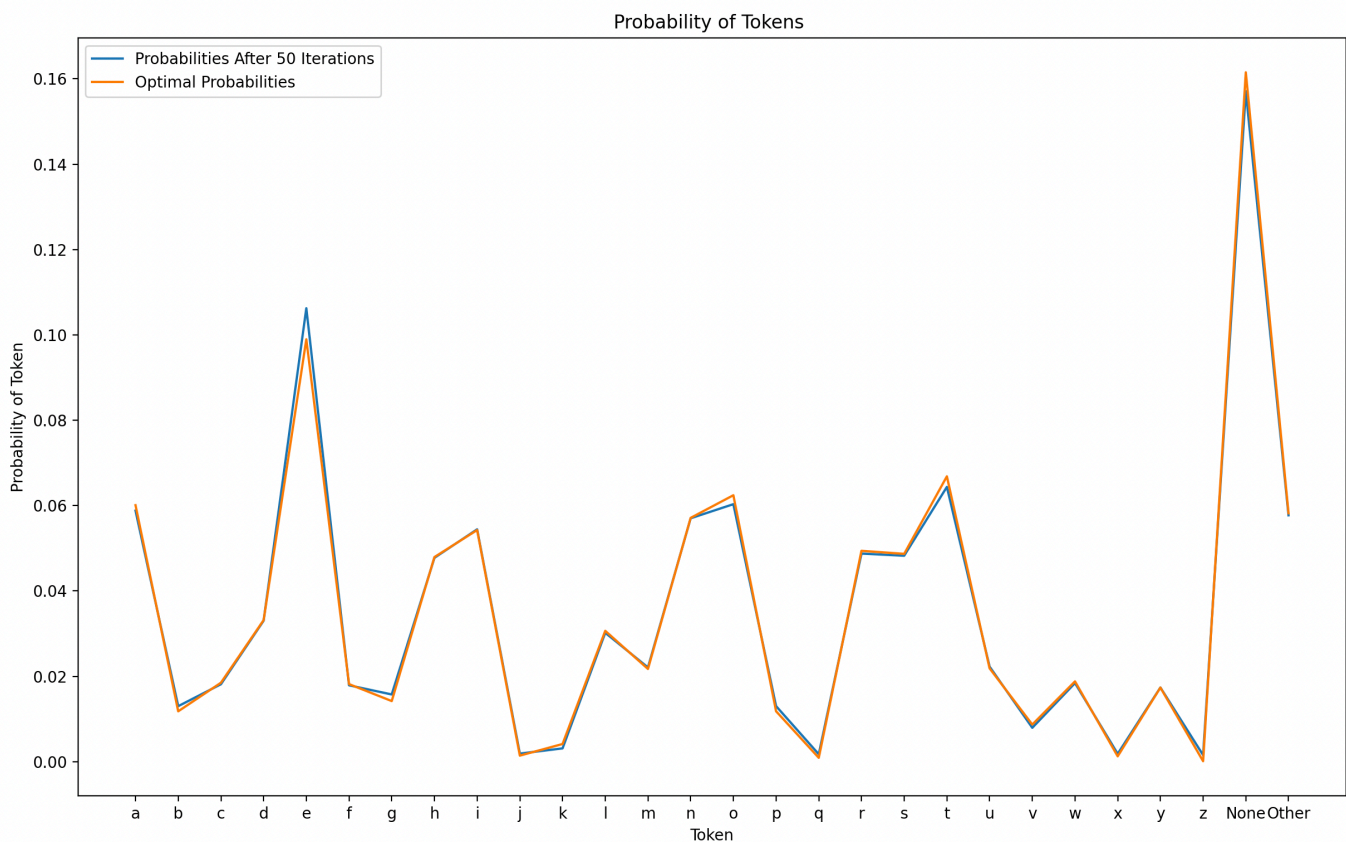


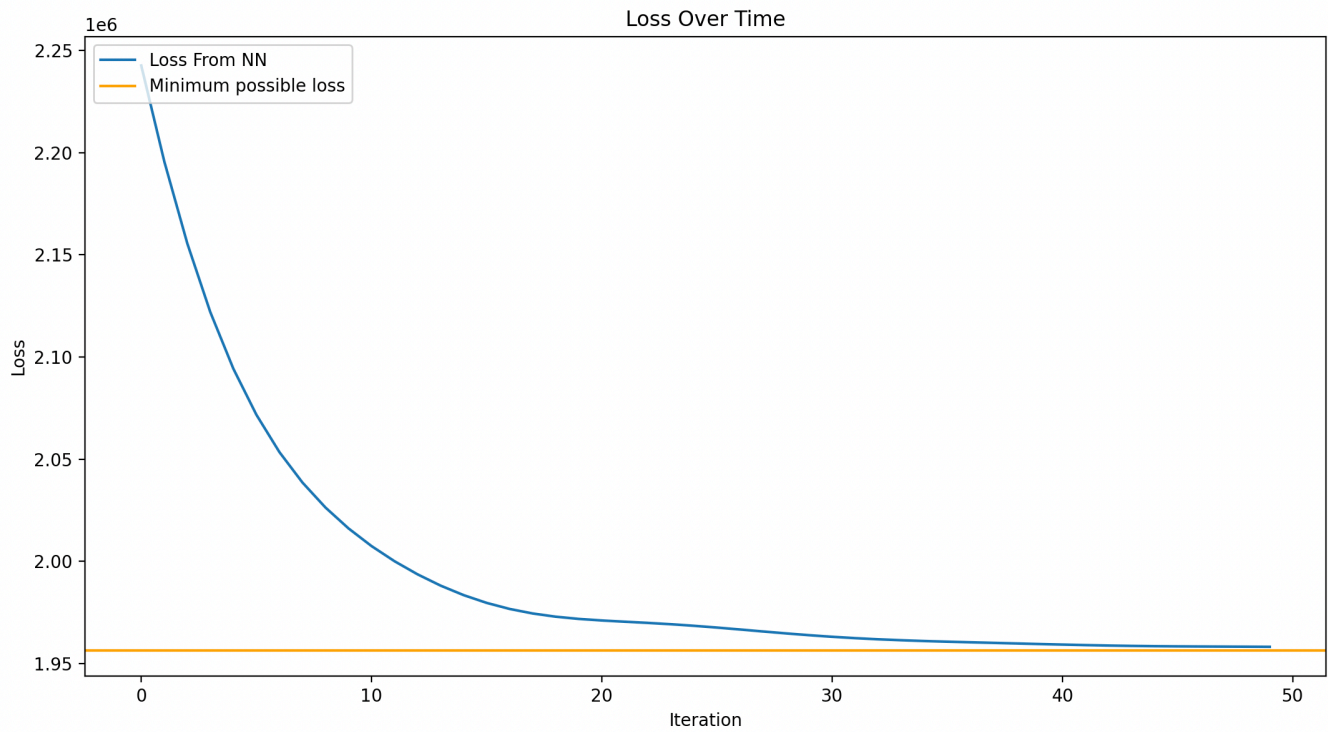
Q1. This neural network attempts to find the optimal token probabilities (these are the weights) for a unigram model trained on a Jane Austen text. The input of the model is the tokens (considered to be individual characters) of the Jane Austen text as a one-hot encoded $V \times T$ array, where V is the length of the vocabulary (made up of the alphabet, an empty string, and an additional token which represents any other characters found in the text and not captured by the previous elements of the vocab) and T is the number of tokens in the training text. The weights (probabilities for each individual token in the vocab) are initialized to 1, normalized so they can form a probability distribution, and logged; then in each iteration, the weights are multiplied by the actual tokens that appear in the text (the one hot encoding array/embedding) to get a sequence of logged probabilities. Summing these gives you the log of the probability of the whole sequence of tokens (the whole training text) occurring. We want this value to be as close to 0 as possible (because we want the probability of the sequence to be as high, aka as close to 1 as possible) and we know through the analytical approach that that means the optimal probability of each token is the count of the token / length of the sequence. The neural network is basically trying to get to this set of probabilities by minimizing the loss (which is just representing how far away the $\log(\text{probability of sequence})$ is from 0) in each iteration. The output of each iteration is the $\log(\text{probability of the sequence})$.

Q2. Visualization of final token probabilities

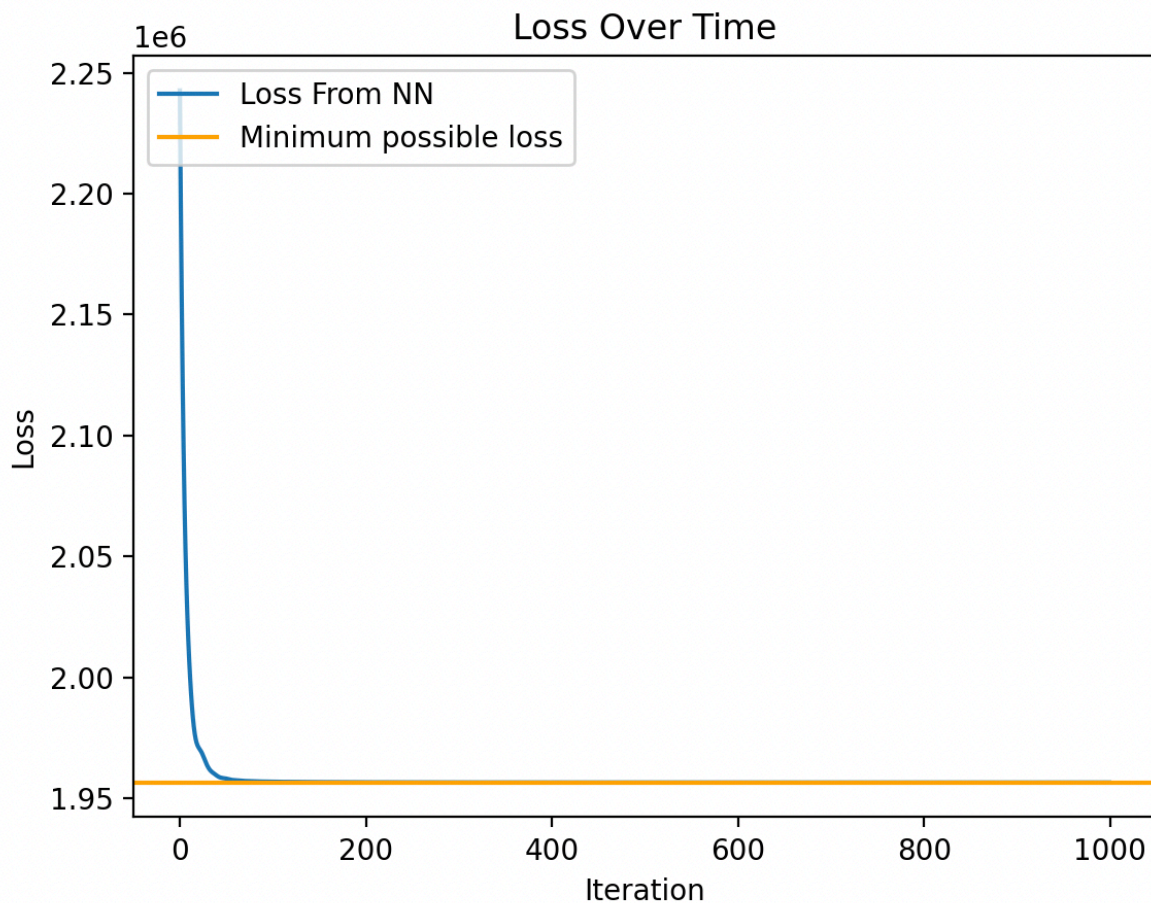


Q3. Visualization of loss over 50 iterations (learning rate = 0.1)

The loss converges to the minimum possible loss pretty closely after 50 iterations.



Visualization of loss over 1000 iterations (learning rate = 0.1)



Q4. Document classification

One approach is to use this same neural network to figure out unigram probabilities for various different texts that belong to different classes. Then you would end up with a unigram vector for each class. In order to classify a new text you could figure out its unigram probabilities in the same way and then figure out the difference in unigram vectors between the new text and the other texts that belong to different classes to see which vectors are the most similar. Based on that you can classify the new text as belonging to the class which has the most similar vector.

Another approach could be to add an additional set of weights for each document class that you are considering as a possible class label in your classification model (for ex. if you are doing binary classification, you need two weight vectors, one for each class). So in the code you would need to initialize another vector similar to self.s. Each vector of weights would be the length of your vocabulary, where the vocabulary includes all of the words in all of the documents you will be using in training. Your input data would now be multiple documents from multiple authors, each with their tokens still one hot encoded, as opposed to just one text. The documents would have to be labeled with their appropriate class. You would need to

change the loss function so that it appropriately evaluates how far the predicted probability of the training data under a certain class (which would be the training data matrix * the weights vector for that class, to get you the log token probabilities vector under that class, and then a sum of those to get the probability of the sequence aka that document under that class) is from the actual class label. Cross entropy loss for example could be used for that. The output of the neural network is ultimately these probabilities of the sequence belonging to a certain class (log softmax could be used to ensure the probabilities of all the classes together sum to 1).